First, we review the architecture of a computer, which is the basis of virtualization technology.

Computer architecture

Since the 1940's, computers have 5 classic components

- **Input** devices: Keyboard, mouse, cartridge, DVD …
- **Output** devices: Display, printer …
- **Storage** devices
  - Volatile **memory**: RAM (Random Access Memory)
  - Permanent **storage**: Magnetic, Optical, and Flash disks, …
- Datapath, Control unit } Together, they are called the **Processor**

In the new concept of a computer, the **network** also is a new component.

The Central Processing Unit (**CPU**) loads programs from the Storage to the RAM and executes in **linear fashion** a succession of instructions (now you can call the programs processes). The "words" that the CPU understands belong to an **Instruction Set**.

An **Operating System** is a special program that controls the hardware and manages the **life-cycle of processes**. Several processes can run concurrently on the same HW, which is shared thanks to an OS that implements CPU time-sharing (**Multitasking**).

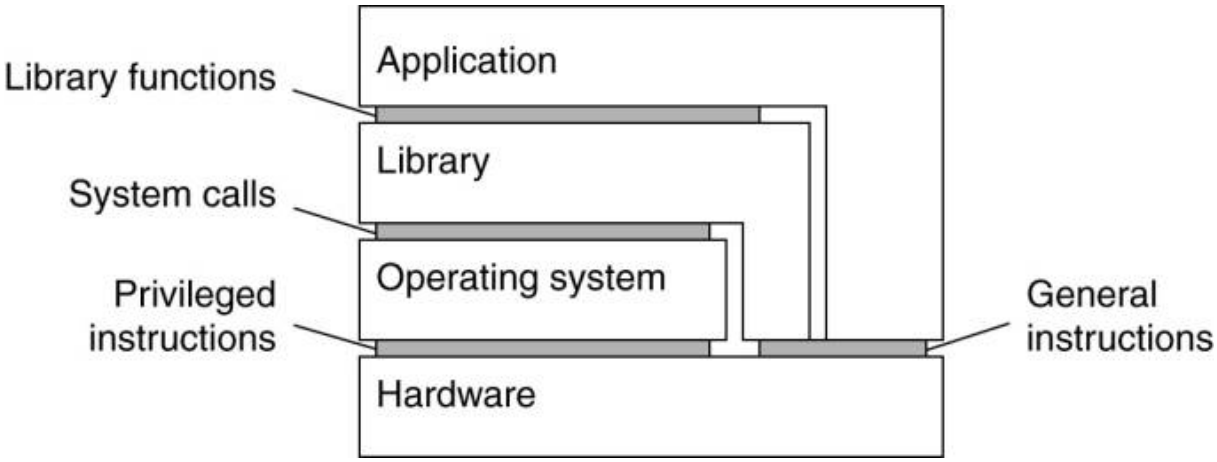


Figure 1: Layers of instructions and interaction with the hardware

We divide instructions by the level of abstraction:

- **Unprivileged machine instructions** (General instructions): an interface to the bare hardware that is available to any program
- **Privileged instructions**: provide an interface to the hardware for the OS
- **System calls**: provide an interface to the operating system for applications
- **APIs**: high-level programming endpoints that may exploits OS system calls

Emulation

Emulation is a technique where one computer system (the **host**) **mimics the hardware and instruction set** of a different computer system (the **guest**). It **translates** instructions from the guest architecture into instructions the host can execute, **recreating the complete behavior of the original system in software**.

Virtualization and Virtual Machines

Virtualization is a technology that creates **abstracted, logical instances of physical computing resources**. It uses a **hypervisor** (or Virtual Machine Monitor) to partition physical hardware and create multiple isolated virtual environments called **Virtual Machines** (VMs). Each VM runs its own operating system and applications, believing it has dedicated hardware, while sharing the underlying physical resources, i.e., the ISA (Instruction Set Architecture).

Aspect	**Virtualization**	**Emulation**
Architecture	Same ISA (e.g., x86 on x86)	Different ISA (e.g., ARM on x86)
Performance	Near-native speed	Significantly slower
Translation	Minimal/no instruction translation	Full instruction translation required
Use case	Server consolidation, cloud computing	Legacy system preservation, cross platform testing
Example	VMware ESXi, KVM, Hyper-V	QEMU (in full emulation mode), DOSBox, game console emulators

Definition 1: Virtual machine

Virtual machines are created and managed by a **Hypervisor** (a.k.a. Virtual Machine Monitor) Examples: VirtualBox, KVM, VMware and Xen.

We can further distinguish between two types of virtual machines:

- **System virtual machines**: Provide a complete system platform that supports the execution of a full operating system (e.g., VMware, VirtualBox, KVM).
- **Process virtual machines** (also application virtual machines or managed runtime environments): Designed to run a single program, providing a platform-independent programming environment (e.g., Java Virtual Machine, .NET Common Language Runtime). They only virtualize the resources needed to ecevute a specific application.
  - **Process-level abstraction**: Virtualizes at the application layer, not the hardware layer
  - **Single application**: Each VM instance runs one application or process
  - **Platform independence**: “Write once, run anywhere” - code runs on any platform with the VM
  - **Managed execution**: Provides runtime services like memory management, garbage collection, and security
  - **Intermediate representation**: Applications are typically compiled to platform-independent bytecode, which the VM interprets or JIT-compiles

From here on, we will focus on system virtual machines, which are the most relevant for cloud computing.

Definition 2: Hypervisor

Protection rings

Modern CPUs define protection levels (rings) from Ring 0 (highest privilege) to Ring 3 (lowest privilege).

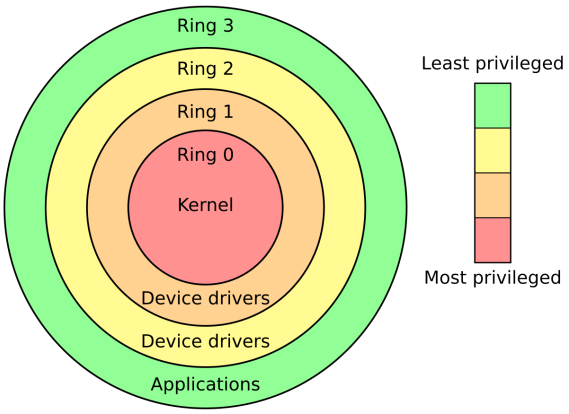


Figure 2: Protection rings

Modern CPUs define protection levels (rings) from Ring 0 (highest privilege) to Ring 3 (lowest privilege). Operating Systems kernels expect to run at Ring 0 to execute privileged instructions directly on hardware. Hypervisors need to intercept these instructions: early solutions used:

- trap-and-emulate.

> In a virtualized environment, the guest OS believes it has full control over the hardware, but it actually runs in a deprivileged mode (typically user mode or a less privileged ring). When the guest OS attempts to execute a privileged instruction, the CPU generates a trap (exception), transferring control to the hypervisor, which then emulates the instruction's behavior in a safe, virtualized manner.

- binary translation.

> Binary translation is a virtualization technique that dynamically scans, analyzes, and rewrites guest code at runtime to replace problematic instructions with safe equivalents that can be executed in a virtualized environment. It was the solution VMware developed to virtualize x86 architecture before hardware virtualization support existed.

Hardware-assisted virtualization (Intel VT-x, AMD-V) introduced new CPU modes that allow guest OSes to run as if they were in Ring 0, while still being controlled by the hypervisor. CPUs add a new mode that lets the **hypervisor run in "root mode"** while the **guest OS runs in "non-root mode"**. When the guest tries to execute a privileged instruction, the CPU automatically traps it and hands control to the hypervisor (Privileged instruction handling), but also:

- Efficient context switching between guest and host environments.
- Memory virtualization support
- I/O virtualization technologies like Intel VT-d and AMD IOMMU allow safe and efficient device sharing between VMs.

Virtualization of resources

Virtualization can be applied to various resources, here we show the most relevant for cloud computing.

Memory virtualization

Memory management is a critical challenge in virtualization: guest OS maintains its own page tables assuming it owns physical memory.

Definition 3: **Shadow page tables**

Definition 4: **Hardware-assisted page tables**

These technologies drastically improved performance, enabling scalable virtualization in cloud data centers.

I/O virtualization

Input/Output devices must also be virtualized for guest systems. We have difference approaches:

- **Device emulation**: hypervisor emulates hardware devices, highly portable but slow.
- **Paravirtualized drivers** (e.g. VirtIO): guest OS communicates directly with hypervisor through optimized interfaces, achieving better performance.
- **Direct device assignment with RS-IOV** (single root I/O virtualization): near-native performance by partitioning PCI (Peripheral Component Interconnect) devices, at the cost of flexibility and migration limitations.

Network virtualization

Network virtualization abstracts physical networking resources (switches, routers, links) into Software-Defined logical Networks (SDN). It allows multiple isolated virtual networks to coexist on the same physical infrastructure, enabling multi-tenancy in cloud data centers. **Virtual switches** (Linux bridge, Open vSwitch) **connect VMs or containers within each host** as if they were plugged into a real switch. **Overlay networks** (e.g., VXLAN, GRE tunnels) interconnect these virtual switches **across hosts, creating logical topologies independent of the underlying physical network**.

The benefits of network virtualization include:

- strong isolation
- programmability through SDN controllers
- elastic provisioning
- simplified management of large-scale infrastructures.

Advantages of virtualization

- **Unified vision of physical resources**
  - resources can be on different physical machines or on the network
- **Centralized management** of hardware and software
- **Workload isolation**. Programs are executed inside the VM and are isolated w.r.t. to the host or other VMs
  - Security
- **Consolidation** of virtualized items
  - Optimization of physical resource usage
- **Easy migration** of virtualized items
- Easy way to implement **Redundancy**.
  - Virtualized items can be easily copied

Workload isolation

Through virtualization, workload isolation is achieved since **all program instructions are fully confined inside a VM**, which leads to improvements in security.

Reliability achieved because software failures inside one VM do not affect others.

Performance control is attained since execution of one VM should not affect the performance of another VM.

Workload migration

Workload migration, a.k.a. **application mobility**, improves:

- hardware maintenance,

- performance,
- fault tolerance.

It is done by allowing the guest OS within a VM to be **suspended, migrated to a different platform**, and **resumed immediately** or **preserved** to be restored at a later date. A VM's state includes a full disk, configuration files, and an image of its memory (RAM).

Consolidation
Virtualization makes it possible to consolidate several virtual machines onto a single physical system, reducing the **total cost of ownership** (TCO), which is an estimate of all the direct and indirect costs involved in acquiring and operating a product or system over its lifetime.

Taxonomy of hypervisors

We can divide hypervisors into two main categories:

- **Bare-metal hypervisors** (Type 1):
  - run directly on the host's hardware to control the hardware and manage guest operating systems. Examples: VMware ESXi, Microsoft Hyper-V, Xen.
  - The hypervisor communicates directly with system hardware rather than relying on a host operating system.
  - The hypervisor is basically a lightweight OS.
  - **advantages**:
    - Improved I/O performance.
  - **Disadvantages**:
    - More complex to set up and manage.
    - It depends on the hardware platform.
- **Hosted hypervisors** (Type 2):
  - run on a conventional operating system just as other computer programs do. Examples: VMware Workstation, Oracle VirtualBox, Parallels Desktop
  - It allows users to run various guest operating systems side by side to their own applications.
  - **Advantages**:
    - Ease of installation and configuration
    - Unmodified Host OS & Guest OS
    - Run on a wide variety of computers
  - **Disadvantages**:
    - Performance overhead due to the additional layer between the hardware and the virtual machines.

Virtualization techniques

There are several virtualization techniques available:

- **Software Virtualization**
- **Hardware Assisted Virtualization**
  - The hardware implements specific technologies to support virtualization
  - Full virtualization and Para virtualization can exploit it for performance

Here we focus on software virtualization techniques.

Full Virtualization using binary translation
Provide a virtual machine environment that is a full emulation of the underlying hardware.

- The guest OS does not need to be modified
- Guest OS is not aware the existence of the virtual environment

- Each VM is independent of each other

The hypervisor executes **privileged instructions** by dynamically translating the instruction of guest OS into a sequence of instruction appropriate to execute in real hardware (**Binary Translation**). Unprivileged instructions are executed directly.

Para Virtualization (a.k.a. OS assisted Virtualization)
The guest OS is modified to be aware that it is running in a virtualized environment. Instead of trying to **fully emulate hardware** (as in full virtualization), the guest OS uses **special calls** to interact directly with the hypervisor. These calls are usually called **hypercalls**.
- the guest os are aware of the hypervisor and use special calls to interact with it
- performance boost
- seamless integration (mouse, keyboard, screen)

paravirtualized drivers are optimized, virtualization- aware device drivers that replace slow device emulation with direct, efficient communication between the guest OS and the hypervisor. Example: **VirtIO** (KVM/QEMU): standard for paravirtual I/O devices (network, block storage, etc.).

OS-Level virtualization (Containerization)
The host operating system itself provides multiple isolated execution environments, known as **containers** (or zones, jails, etc.), rather than emulating full hardware machines. Unlike hypervisor-based virtualization, OS-level virtualization shares the **same kernel between all containers**. Each container runs as a set of isolated processes, with its own view of the filesystem, process table, network stack, and resource quotas. **The virtualization capability is provided by the host OS**, which performs all the functions of a fully virtualized hypervisor
- Linux Container (Docker, LXD…)